

PNPI • Architecture backend

Version : 1.0 (lot 79.4) **Periode** : avril 2026 **Audience** : developpeurs backend, architectes, equipe SecOps, auditeurs ministeriels.

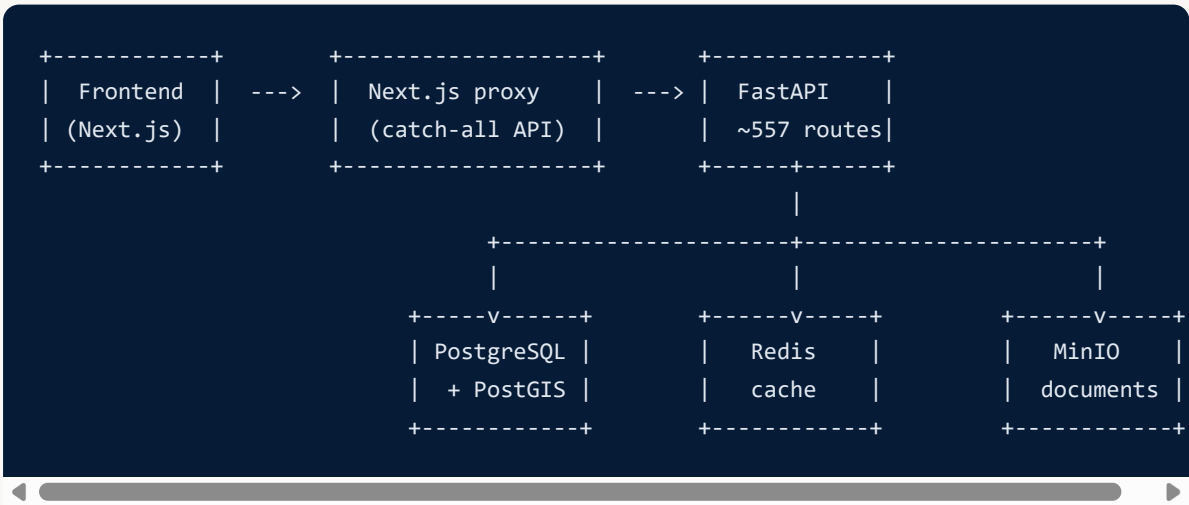
*Ce document est la **carte du territoire backend** de la Plateforme Nationale de Pilotage Industriel (PNPI). Il couvre l'organisation du code, le modele de donnees, la pile applicative et les patterns defensifs a respecter pour intervenir sans regression.*

1. Vue d'ensemble

1.1 Stack et runtime

COMPOSANT	CHOIX	VERSION
Langage	Python	3.12
Framework HTTP	FastAPI (ASGI)	>=0.115
ORM	SQLAlchemy 2.x	>=2.0.36
Migrations	Alembic	>=1.14
Base SQL	PostgreSQL + PostGIS	16 / 3.4
Cache	Redis (fallback in-memory)	5.x
Auth tokens	python-jose (JWT HS256)	3.3
Hash mdp	passlib + bcrypt	1.7 / 4.0
Chiffrement	cryptography (Fernet AES-CBC+HMAC)	>=43
Rate limit	redis (sliding window) + memory	-
Observabilite	logging JSON + Prometheus metrics	-

1.2 Topologie



L'app monte avec **environ 557 routes** declarees au lancement.

1.3 Middleware stack (ordre d'execution)

```
GZip
-> CSRF (origin validation)
-> Timeout (30s)
-> RequestSizeLimit (10/50 MB)
-> Metrics (Prometheus)
-> Correlation (X-Request-ID)
-> CORS
-> RateLimiter (par IP/endpoint)
-> SecurityHeaders (CSP, HSTS, etc.)
```

L'ordre n'est pas anodin : GZip en premier pour decompresser, CSRF avant le timeout pour rejeter rapidement les requetes hostiles, etc.

2. Organisation du code

```
backend/app/
  main.py          # FastAPI app, lifespan, middleware stack,
                   # routes legacy, seeds, boucle SLA en background
  api_v1.py        # Router /api/v1/* (versioning)
  config.py        # Settings (env vars PNPI_*) + validation
  database.py      # SQLAlchemy engine, pool, retry, helpers UTC
  core/            # 30+ modules transverses (auth, audit, cache,
                   # rate limiter, csrf, sanitize, encryption, ...)
  models/          # ORM (base.py + core.py + pnpi.py + pilotage.py)
  schemas/         # Pydantic (entree/sortie API + exemples OpenAPI)
  routers/         # 40 modules : un par domaine fonctionnel
  alembic/         # 38 migrations (apres lot 79)
backend/scripts/   # CLI : seed, backup S3, encrypt NIF, smoke tests
backend/tests/     # Pytest (~20 fichiers)
```

3. Modele de donnees

3.1 Tables principales

Domaine PNPI metier (`models/pnpi.py`) :

- `opérateurs_industriels` : entités économiques. PK `id` , `nif_gabon` (clair) + `nif_gabon_encrypted` (Fernet at-rest, lot 79.2). Soft delete via `deleted_at` .
- `agreements_ati` : Agreements Techniques Industriels. PK `id` , FK `opérateur_id` , statut/etape (etat metier), `instructeur_username` , `created_by` (opérateur propriétaire), SLA en jours.
- `ati_transitions` : audit metier des changements d'etat ATI.
- `ati_appeals` : recours post-rejet (lot 77).
- `ati_comments` , `ati_tags` , `ati_checklist_items` , `ati_reminders` : outillage instructeur autour d'un dossier.
- `inspections_conformite` + `inspection_photos` : inspections terrain.
- `documents_dossier` + `document_versions` : pieces ATI.
- `messages` , `annoncements` , `polls` , `conventions` , `delegations` , `instructor_ratings` , `operator_feedback` : collaboration interne.

Domaine technique / pilotage (`models/core.py` , `models/pilotage.py`) :

- `units` , `declarations` : declarations production legacy (compatibilite avec la v1 du systeme).
- `trace_batches` : tracabilite avec QR.
- `field_reports` : rapports terrain (mobile inspecteur).
- `user_accounts` : comptes utilisateurs (`username` PK, hash bcrypt, TOTP, backup codes, province, lock-out automatique apres 5 echecs).
- `notifications` , `notification_preferences` : push + e-mail.
- `audit_events` : trail securite (actor / action / target / details).
- `login_history` : journal des connexions.
- `refresh_tokens` : rotation et revocation des refresh JWT.
- `push_subscriptions` : Web Push API (lot 76).
- `project_dossiers` : dossiers strategiques (vue ministre).

3.2 Contraintes et patterns

- Toutes les `datetime` sont `DateTime(timezone=True)` . En SQLite (dev), elles sont stockees naive ; le helper `as_utc()` de `app.database` doit etre utilise avant toute comparaison a `now_utc()` . C'est un **pattern obligatoire** documente dans `CLAUDE.md` .
- Les FK pointent vers `user_accounts.username` (clef texte) et non un id numerique. Choix historique pour faciliter l'audit lisible humain.
- Soft delete : pour l'instant uniquement sur `opérateurs_industriels` .
- Index : voir `docs/architecture/backend-perf-audit.md` pour la cartographie complete et la migration 38.

3.3 Migrations

- 38 migrations sequentielles, format `YYYYMMDD_NN_description.py` .
 - L' `alembic_version` est gere automatiquement.
 - Les migrations recentes sont :
 - 32 : composite indexes
 - 33 : soft delete operateurs
 - 34 : geo sur photos inspection
 - 35 : push subscriptions
 - 36 : recours ATI
 - **37** : `nif_gabon_encrypted` (lot 79.2)
 - **38** : index perf manquants (lot 79.3)
-

4. Modules core/ (transverses)

MODULE	ROLE
<code>auth.py</code>	JWT (access + refresh), bcrypt, lock-out, fallback fake_users_db dev
<code>audit.py</code>	<code>write_audit_event()</code> (toujours commit avec la session courante)
<code>cache.py</code>	Redis async + fallback dictionnaire in-memory si Redis down
<code>rate_limiter.py</code>	Sliding window (key par IP+endpoint), backed Redis
<code>correlation_middleware.py</code>	X-Request-ID injecte dans les logs JSON
<code>csrf.py</code>	Validation Origin/Referer pour les mutations
<code>sanitize.py</code>	XSS scrub + type Pydantic <code>SanitizedStr</code>
<code>timeout_middleware.py</code>	Coupe les requetes >30s avec 504
<code>request_size_limit.py</code>	10 MB body / 50 MB upload
<code>upload_validation.py</code>	Mime + magic number + taille
<code>encryption.py</code> (lot 79)	Fernet at-rest pour champs sensibles (NIF). MultiFernet rotation.
<code>webhooks.py</code>	Sortie outbound avec retry exponentiel
<code>feature_flags.py</code>	Toggles via <code>PNPI_FF_*</code> (ex: <code>PNPI_FF_GRAPHQL_API</code>)
<code>health_score.py</code>	Score sante global 0-100 (uptime, latence, queue, DB)
<code>error_handlers.py</code>	Pydantic 422 -> reponse francisee, 500 capture stack tracee
<code>logging_config.py</code>	Structured JSON logs (production) ou texte (dev)
<code>metrics.py</code>	Prometheus middleware + scraping <code>/metrics</code>
<code>pagination.py</code>	<code>PaginationParams</code> , <code>PaginatedResponse</code> reutilisables
<code>tenant.py</code>	Filtrage par province (multi-tenant CEMAC future-proof)

MODULE	ROLE
<code>signature.py</code>	Signature electronique inspections + certificats
<code>scoring.py</code>	Score de conformite operateur
<code>risk_assessment.py</code>	Risk profile ATI (delai, criticite secteur, historique)
<code>decision_engine.py</code>	Aide a la decision automatisee (lot 76)

5. Routers (40 modules)

Regroupees par domaine :

5.1 Authentification & comptes

`auth.py` (login/refresh/logout, captcha, change-pwd, login history, preferences notifications, badges, daily digest), `totp.py` (2FA TOTP + backup codes), `api_keys.py` (cles longues duree pour integrations).

5.2 ATI / dossiers

`ati.py` (CRUD + workflow + helper `check_ati_access`), `documents.py` (upload/download dossier), `appeals.py` (recours, lot 77), `conventions.py`, `delegations.py`, `checklists.py`, `doc_versions.py`, `templates.py`.

5.3 Operateurs et pilotage

`operateurs.py` (CRUD, scores, timeline, soft delete, import CSV), `inspections.py` (rapports terrain + photos geo), `pnpi_dashboard.py` (KPIs ministre/directeur), `pilotage.py` (vue agregées legacy).

5.4 Cartographie & exports

`geo.py` (geocodage Gabon), `heatmap.py` (densite ATI par province), `exports.py` (CSV/XLSX/PDF), `reports.py` (PDF rapports executifs), `scheduled_reports.py` (cron des rapports).

5.5 Search & discovery

`search.py` (full-text local), `open_data.py` (stats publiques k-anonymity).

5.6 Communication

`messages.py` (inbox interne), `notifications.py` (web + push), `announcements.py`,
`feedback.py`, `polls.py`, `chat.py` (assistant LLM).

5.7 Administration

`admin.py` (impersonate, backups, raci, simulateur), `health.py` (liveness/readiness + DB/Redis),
`integration.py` & `integration_health.py` (webhooks externes, surveillance).

5.8 Realtime & autres

`ws.py` (WebSocket pour notifications live), `workflows.py` (moteur), `reminders.py`, `notes.py`,
`calendar.py`, `units.py`, `push.py` (Web Push lot 76), `graphql_api.py` (feature flag,
optionnel).

6. Conventions et patterns defensifs

6.1 RBAC

Six roles (`Role` `StrEnum` dans `core/auth.py`) : `admin` | `ministre` | `directeur` | `instructeur` | `inspecteur` | `opérateur` .

- Decorateur unique : `Depends(require_roles(...))` .
- Helper centralise `check_ati_access(at_i, current_user)` dans `routers/at_i.py` : tout endpoint `/pnpi/at_i/{id}/*` ouvert a opérateur doit l'appeler — sinon IDOR. Audit RBAC fait : 108 combos, 0 fuite.
- Opérateur voit l'annuaire (raison sociale, secteur, province) sans les NIF/effectifs des concurrents (`routers/opérateurs.py`).

6.2 Datetime UTC

```
from app.database import as_utc, now_utc

# OK
if as_utc(ati.date_decision) >= now_utc() - timedelta(days=1):
    ...

# KO - crash en SQLite (naïve vs aware)
if ati.date_decision >= now_utc():
    ...
```

6.3 Roles : ne jamais comparer en direct

```
# OK
roles = {r.value if hasattr(r, "value") else str(r) for r in current_user.roles}
if "opérateur" in roles: ...

# KO - mix Enum vs str selon source (DB / fake_users_db)
if Role.opérateur in current_user.roles: ...
```

Helper local `_user_role_values(user)` dans `routers/ati.py` est la référence.

6.4 Audit trail systématique

Toute mutation sensible doit appeler `write_audit_event(db, actor=..., action=..., target=..., details=...)` dans la même transaction que la mutation, **avant le commit**.

6.5 Chiffrement at-rest (lot 79.2)

Champ NIF : utiliser `set_nif()` et `nif` (property) sur `OpérateurIndustrieIORM`. La colonne `nif_gabon` reste peuplée en clair durant la phase de transition. Une migration future supprimera la colonne en clair une fois 100% des lignes chiffrées (script `scripts/encrypt_existing_nifs.py`).

7. Tests et CI

7.1 Backend tests

- `pytest` ~20 fichiers, ~93 tests environ.

- `tests/conftest.py` (lot 79.1) :
- force `PNPI_DATABASE_URL` vers `pnpi-test.db` AVANT import de `app.*`.
- normalise les mots de passe `*-dev-password` via env vars.
- reseed `user_accounts` au démarrage de la session pour aligner les hashes bcrypt avec les attendus des tests.
- Les tests qui cassent localement (bug DB state pourri) sont stabilisés par cette fixture `session-scoped`.

7.2 CI

- GitHub Actions : `ci.yml` (lint + test + build) + `cd.yml` (deploy).
 - Ruff (`backend/`), Prettier (`frontend/`), pip-audit, npm audit.
 - Tests en CI passent toujours (DB fraîche par run), tests locaux nécessitent la nouvelle `conftest` pour la même garantie.
-

8. Points de vigilance

ZONE	RISQUE	MITIGATION
<code>routers/ati.py</code>	IDOR si on ajoute un endpoint sans helper	Toujours appeler <code>check_ati_access(ati, current_user)</code>
<code>routers/auth.py</code>	Lockout automatique apres 5 echecs	Reset via admin ou attendre 15 min
<code>core/cache.py</code>	Fallback in-memory non partage entre workers	En prod, exiger Redis (<code>PNPI_REDIS_URL</code> defini)
<code>database.py</code> engine module	<code>engine</code> cree au load -> bind sur 1 seule URL	Tests : surcharger <code>PNPI_DATABASE_URL</code> avant import
<code>main.py:seed_user_accounts</code>	Insert si vide -> hash fige a la 1ere creation	Ne jamais s'appuyer sur seed pour aligner mdp en test
<code>core/encryption.py</code>	Cle perdue = donnees illisibles	Backup chiffre de la cle dans le secret manager + KMS
<code>pnpi.py:OperateurIndustrielORM</code>	Double colonne NIF transitoire	Toujours utiliser <code>set_nif()</code> jamais <code>nif_gabon=</code> direct
<code>routers/admin.py</code>	Impersonate puissant	Audit chaque appel ; banner frontend obligatoire
<code>core/rate_limiter.py</code>	Memory backend = bypass entre process	En prod, Redis obligatoire
<code>models/pnpi.py:datetime</code>	Comparaison naive/aware en SQLite	<code>as_utc()</code> systematique avant compare

8.1 Mots-cles `grep` utiles

```
# Tous les endpoints qui mutent un ATI doivent passer par check_ati_access
rg "check_ati_access" backend/app/routers/

# Tous les endpoints sensibles doivent emettre un audit
rg "write_audit_event" backend/app/routers/

# Comparaisons de datetime (pour traquer les oublis as_utc)
rg "now_utc\\(\\)\\s*[-<>=]" backend/app/

# Toute mutation sur les operateurs doit utiliser set_nif (lot 79.2+)
rg "nif_gabon\\s*=" backend/app/

# Le helper require_roles doit couvrir chaque endpoint
rg "@router\\. (get|post|put|patch|delete)" backend/app/routers/ -A2 | rg -v "Depends\\("
```

9. Bibliotheques externes critiques

LIB	USAGE	RISQUE SI REMPLACEE
<code>python-jose</code>	JWT encode/decode	Compat HS256 + JTI
<code>passlib[bcrypt]</code>	Hash mots de passe	Plages de cost factors a aligner
<code>cryptography</code>	Fernet field encryption + signatures	Key derivation et format incompatible
<code>sqlalchemy</code>	ORM 2.x (mapped_column declarative)	Migration vers 2.x deja faite
<code>alembic</code>	Migrations	Ne pas modifier les revisions deja jouees
<code>redis</code>	Cache + rate limit	Fallback in-memory degrade prod
<code>httpx</code>	Webhooks sortants async	Timeouts a verifier
<code>pyotp</code>	TOTP / 2FA	RFC 6238 strict
<code>pywebpush</code>	Web Push lot 76	VAPID keys a rotater
<code>boto3</code>	Backups S3 / MinIO	Compat S3 v4 sigs
<code>reportlab</code> + <code>qrcode</code>	PDF + QR ATI	Police Inter doit etre embarquee
<code>GeoAlchemy2</code>	PostGIS geographies	Requiert extension postgis activee

Si vous touchez l'une de ces libs, declencher un mini smoke-test :

```
bash scripts/pre-deploy-check.sh
```

10. Ressources et liens internes

- Plan de montée de version : `docs/implementation_roadmap.md`
- Recovery / PRA-PCA : `docs/prp_pca.md`
- Audit perf detaille : `docs/architecture/backend-perf-audit.md`
- ADRs : `docs/architecture/adr-001-...` , `adr-002-...` , etc.
- Conventions code : `CLAUDE.md` (racine repo)
- Skills review : `docs/skills-guide.html`

Fin du document. Pour toute correction, ouvrir une PR `docs:` avec le numero du lot en cours (ex. `docs: lot 80 – corrige section X`).